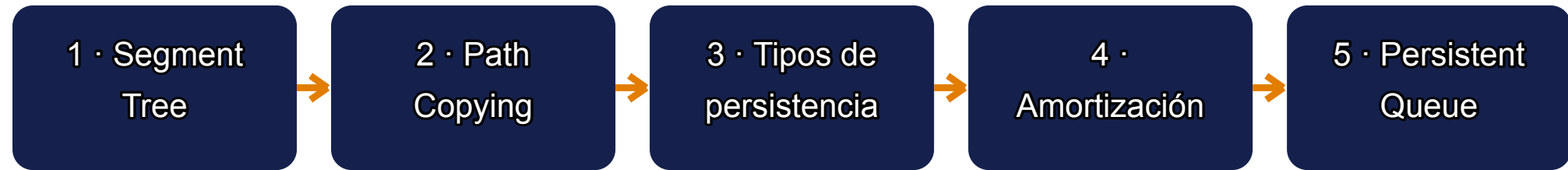


Estructuras de Datos Persistentes

Benjamín Letelier



de segment tree normal a persistente

$O(\log n)$ nodos nuevos por versión, el resto se comparte

Del segment tree normal a su versión **persistente**: guardar toda la historia con solo $O(\log n)$ nodos nuevos por update.

Segment Tree

1 · Seg Tree

2 · Path Copying

3 · Tipos

4 · Amortización

5 · Queue



Segment Tree — ¿Qué es?

Estructura para responder **consultas por rango** y **actualizaciones puntuales** en tiempo logarítmico.

- Range sum query: `sum(l, r)`
- Range minimum query: `min(l, r)`
- Point update: `a[pos] = x`

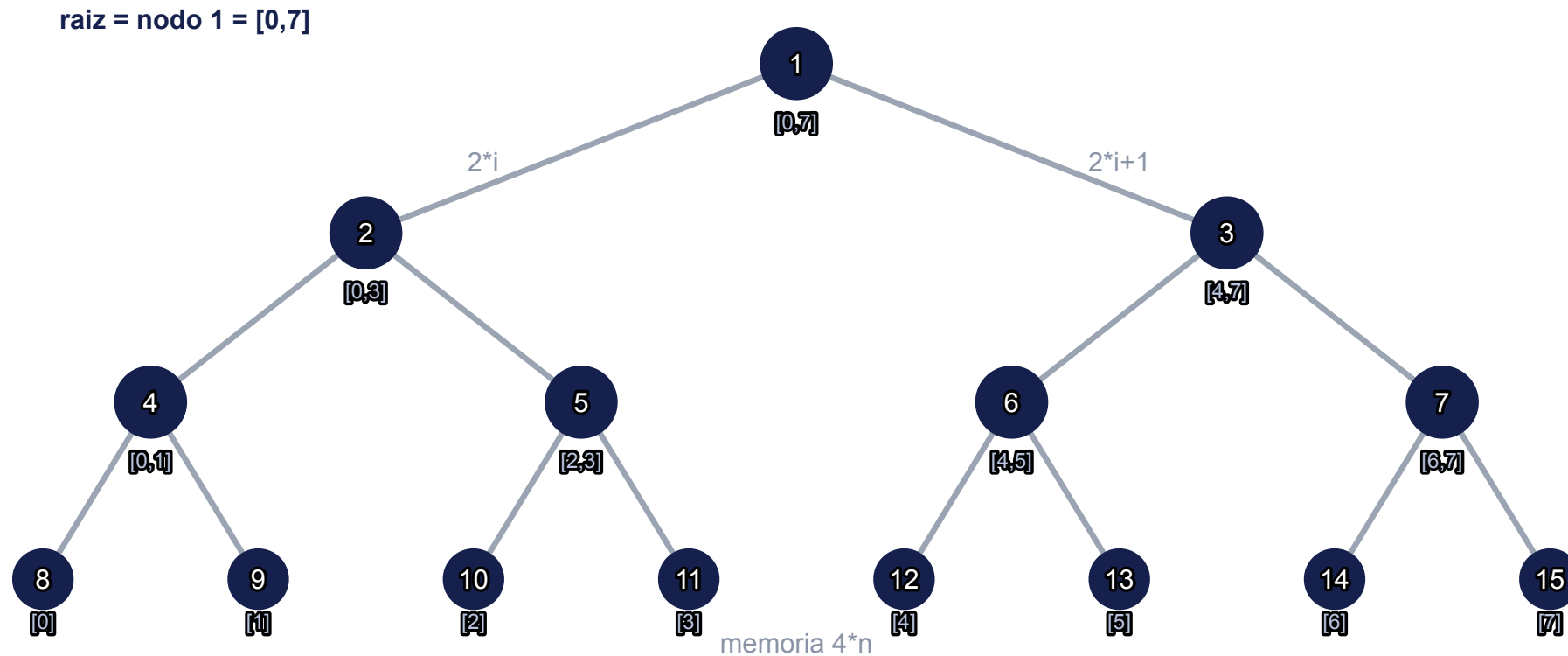


Cada operación en **$O(\log n)$** : ideal cuando hay muchas consultas y updates entremezclados.



Segment Tree — Estructura

Árbol binario donde cada hoja es un elemento del array y cada nodo interno guarda un valor agregado (suma, mínimo, ...).

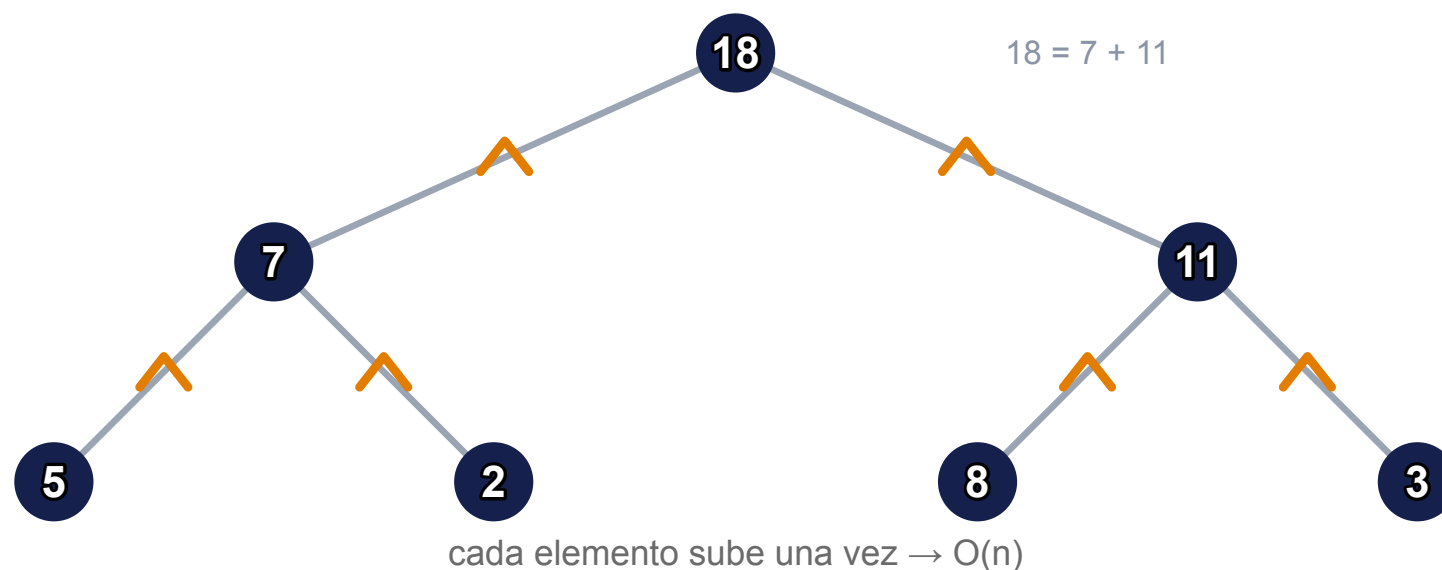


- Raíz (nodo 1): cubre todo el rango $[0, n-1]$
- Hijo izquierdo $2 \cdot i$: mitad izquierda · Hijo derecho $2 \cdot i + 1$: mitad derecha



Segment Tree — Build

Cada elemento sube por el árbol exactamente una vez $\rightarrow O(n)$.



Construcción recursiva *bottom-up*: combinamos los hijos al volver de la recursión.



Segment Tree — Build $O(n)$

```
int build(int i, int l, int r) {  
    if (l == r) return seg[i] = a[l];  
    int m = (l + r) / 2;  
    return seg[i] = build(2*i, l, m)  
                    + build(2*i+1, m+1, r);  
}
```

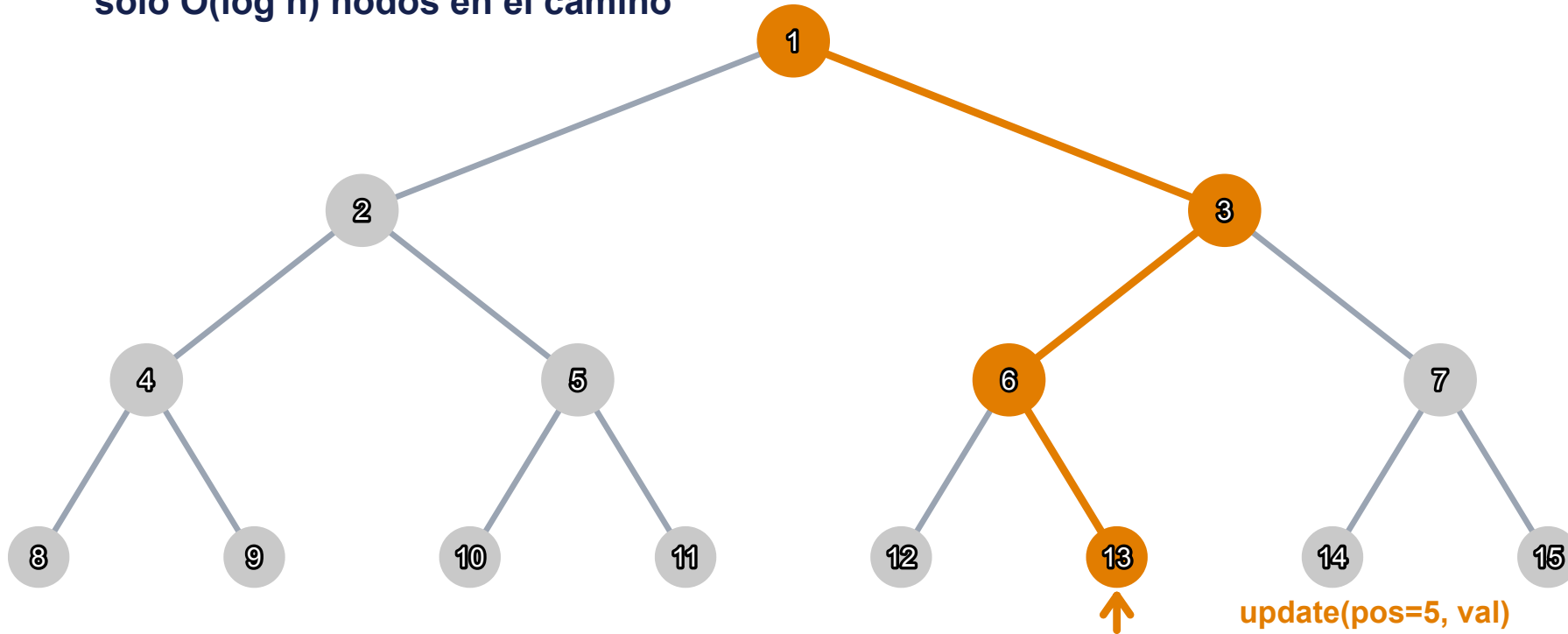
Cada elemento sube por el árbol exactamente una vez $\rightarrow O(n)$.



Segment Tree — Update

Bajamos a la hoja y actualizamos los nodos en el camino de regreso: solo $O(\log n)$ nodos.

solo $O(\log n)$ nodos en el camino



Un update puntual toca únicamente los nodos del camino raíz $\rightarrow$ hoja.



Segment Tree — Update $O(\log n)$

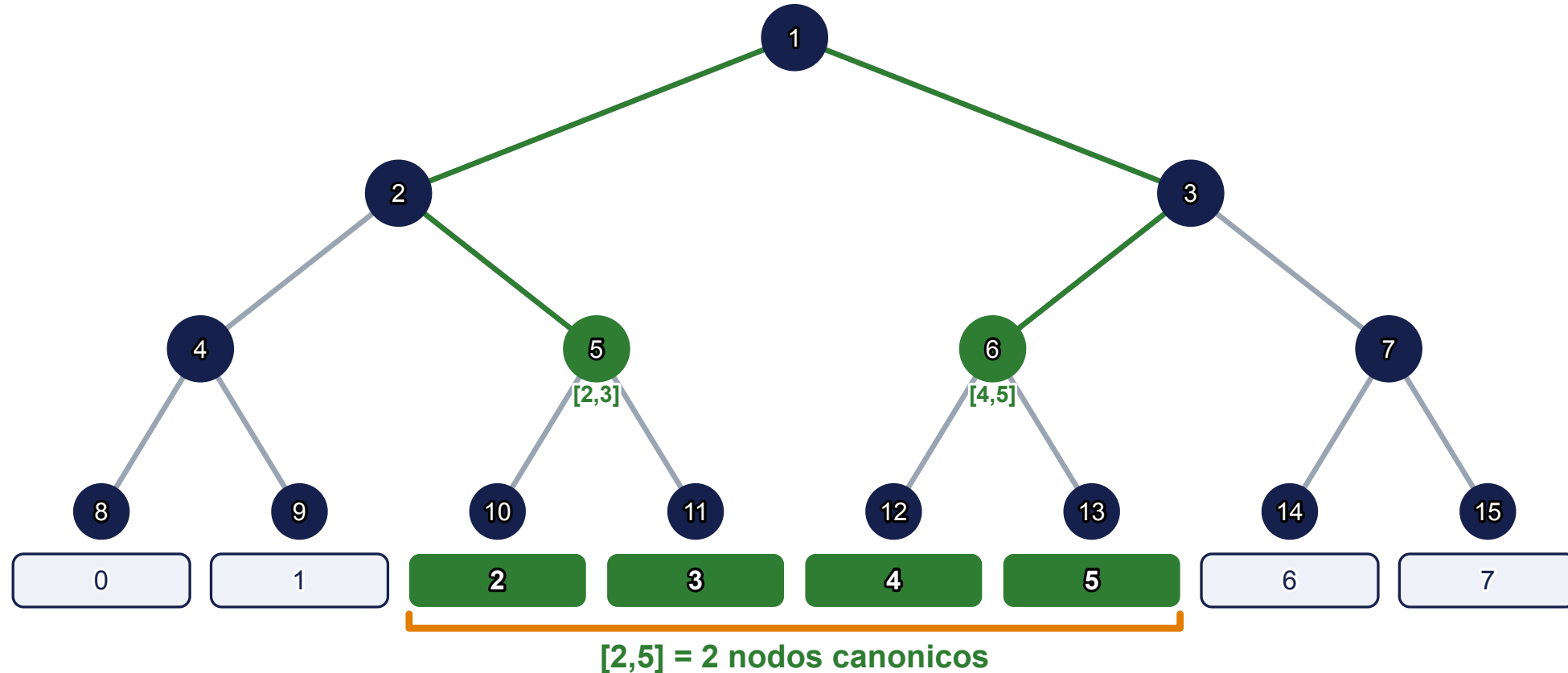
```
void update(int i, int l, int r, int pos, int val) {  
    if (l == r) { seg[i] = val; return; }  
    int m = (l + r) / 2;  
    if (pos <= m) update(2*i, l, m, pos, val);  
    else update(2*i+1, m+1, r, pos, val);  
    seg[i] = seg[2*i] + seg[2*i+1];  
}
```

Solo tocamos $O(\log n)$ nodos por actualización.



Segment Tree — Query

Visitamos solo los nodos cuyos rangos están completamente cubiertos por $[ql, qr]$.



El rango $[ql, qr]$ se descompone en $O(\log n)$ nodos canónicos.



Segment Tree — Query $O(\log n)$

```
int query(int i, int l, int r, int ql, int qr) {  
    if (qr < l || r < ql) return 0;           // sin solapamiento  
    if (ql <= l && r <= qr) return seg[i];    // completamente dentro  
    int m = (l + r) / 2;  
    return query(2*i, l, m, ql, qr)  
           + query(2*i+1, m+1, r, ql, qr);  
}
```



El problema

Cada **update** **sobrescribe** el valor anterior. No hay forma de consultar **versiones anteriores** del arreglo.



el update pisa el valor anterior

Si necesitamos responder "*¿cuál era la suma en $[l, r]$ tras las primeras k operaciones?*" ... necesitamos **persistencia**.

Path Copying

1 · Seg Tree

2 · Path Copying

3 · Tipos

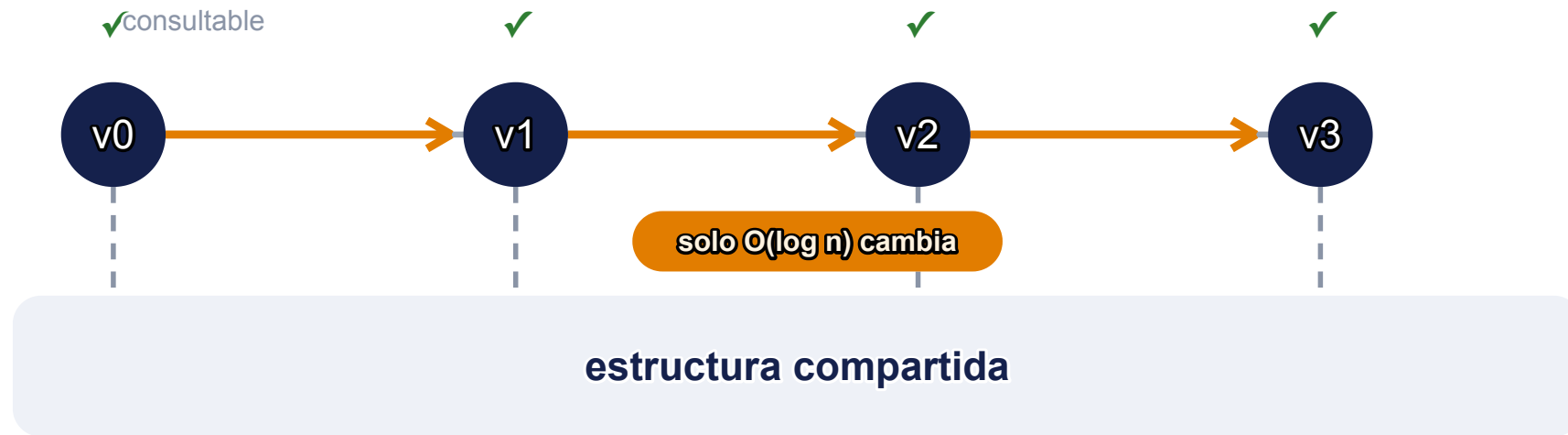
4 · Amortización

5 · Queue



Persistencia — Idea intuitiva

Queremos un *time machine* para nuestra estructura de datos.



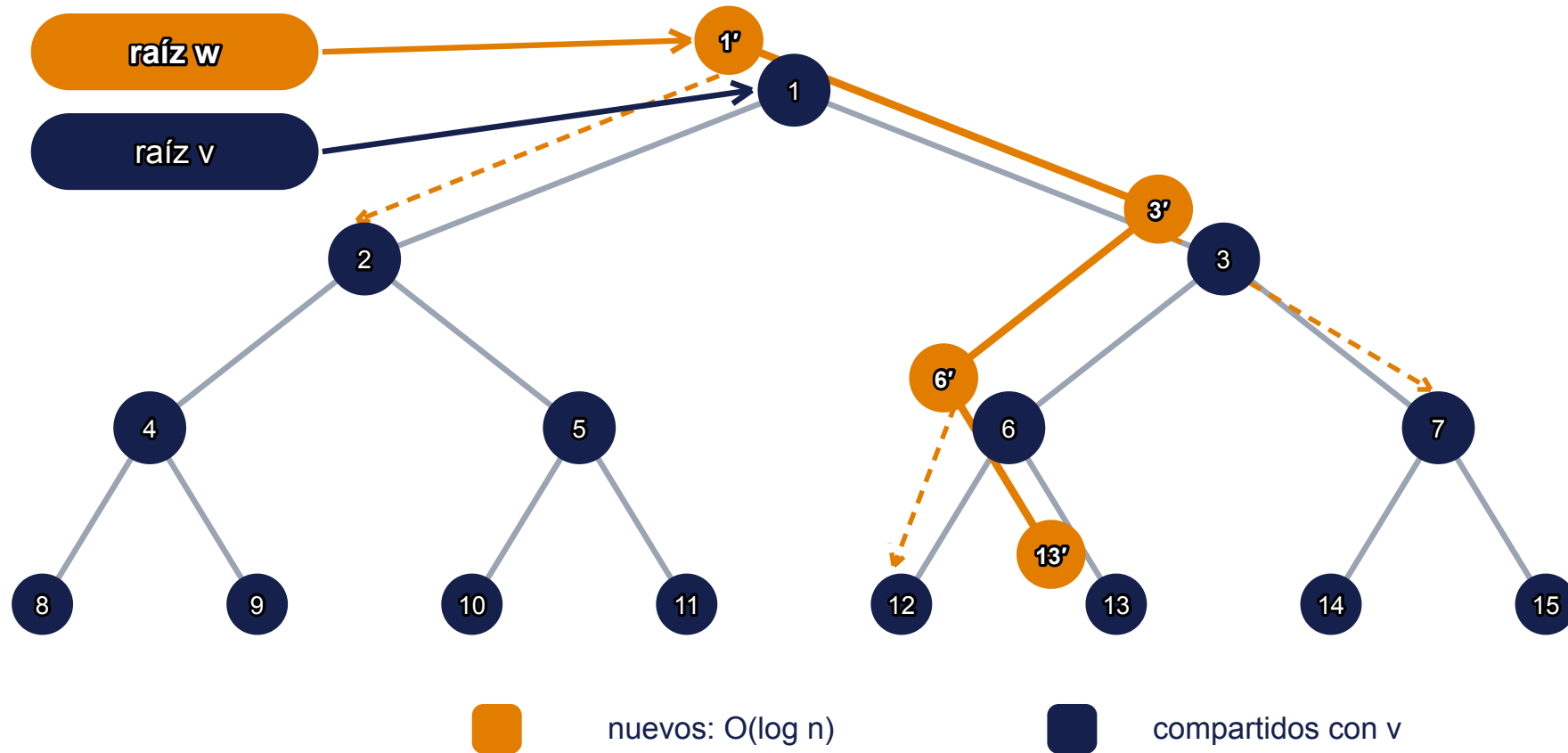
time machine: versiones consultables que comparten

- Cada update genera una **nueva versión**; podemos consultar **cualquier** versión anterior.
- **Observación clave:** cada update solo modifica $O(\log n)$ nodos. El resto del árbol permanece igual.



Segment Tree Persistente — Path Copying

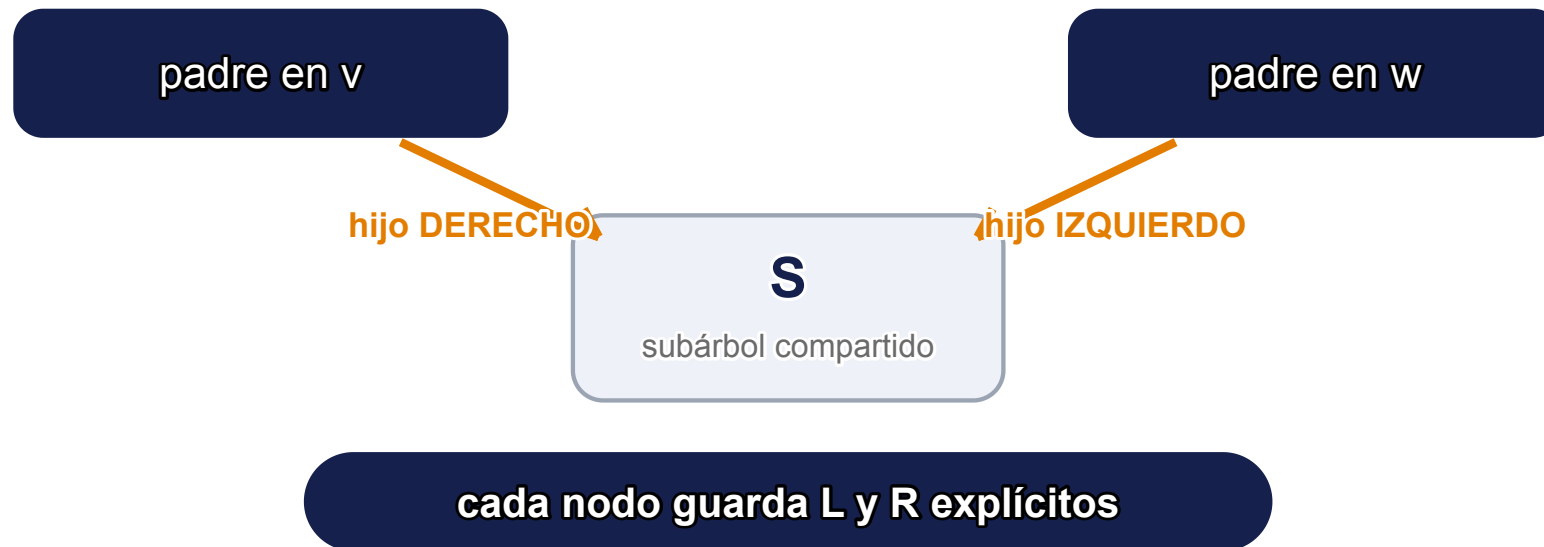
En vez de sobrescribir nodos, creamos nuevos nodos en el camino de la raíz a la hoja modificada.



- Los nodos **fuera del camino** se **comparten** con la versión anterior.

¿Por qué no sirve el array implícito?

En un segment tree normal usamos $2*i$ y $2*i+1$ para los hijos. Con persistencia, **varias versiones comparten subárboles**.

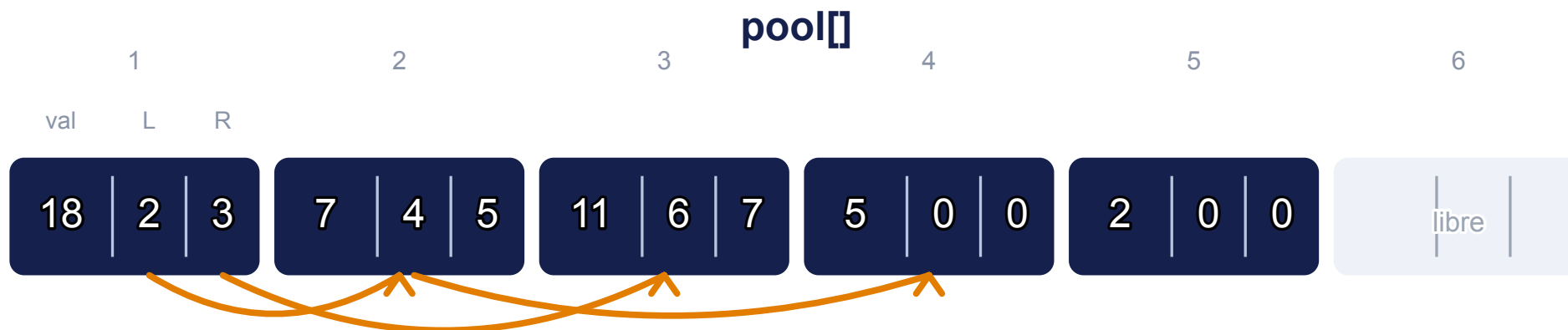


Un mismo nodo puede ser hijo derecho en una versión e izquierdo en otra. **Solución:** cada nodo guarda explícitamente los índices de sus hijos (L , R) en un *node pool*.



Implementación — Node Pool

Un arreglo global de nodos; cada nodo referencia a sus hijos por **índice**, no por posición implícita.



new_node añade en ++node_cnt; hijos por índice

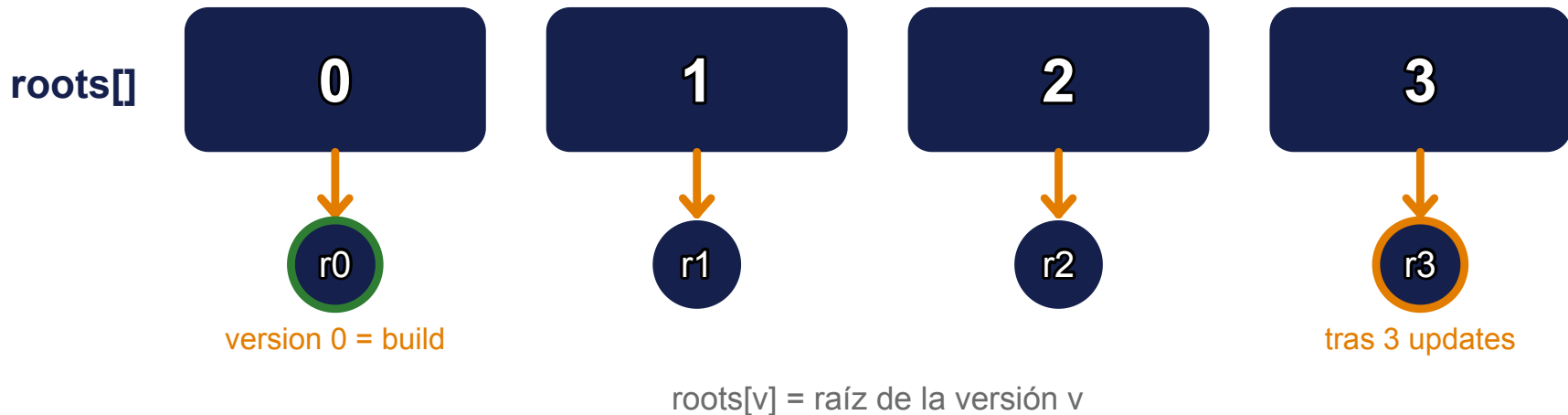


Implementación — Node Pool

```
struct Node {  
    int val;    // valor agregado (suma, min, etc.)  
    int L, R;   // índices de hijos en el pool  
    Node() : val(0), L(0), R(0) {}  
};  
  
Node pool[MAX_NODES];  
int node_cnt = 0;  
  
int new_node(int v = 0, int l = 0, int r = 0) {  
    pool[++node_cnt] = {v, l, r};  
    return node_cnt;  
}
```

Implementación — Array de raíces

Cada versión se identifica por el índice de su **nodo raíz**.

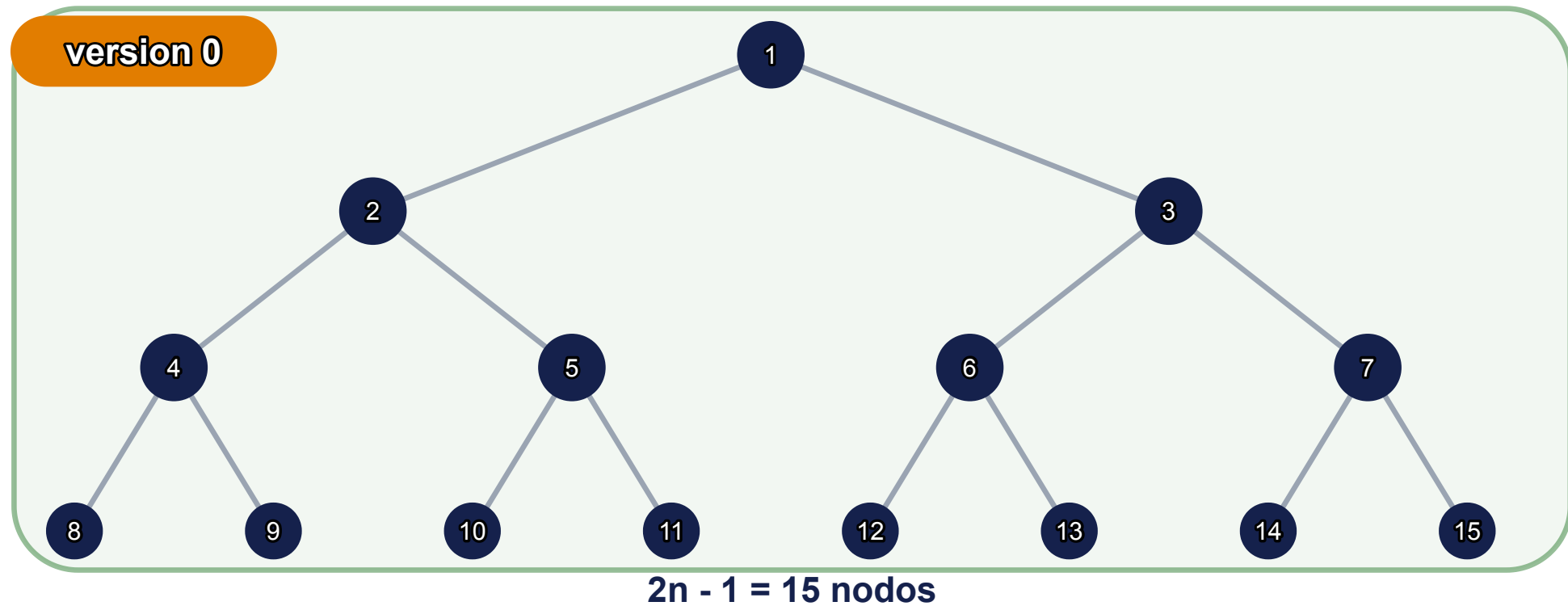


```
int roots[MAX_VERSIONS]; // roots[v] = índice del nodo raíz de la versión v
int version_count = 0;
```

- Versión 0: estado inicial (construida con `build`)
- Versión `v` : después del `v` -ésimo update sobre alguna versión base



Igual que el segment tree normal, pero pidiendo nodos al pool: $2n - 1$ nodos para la versión inicial.





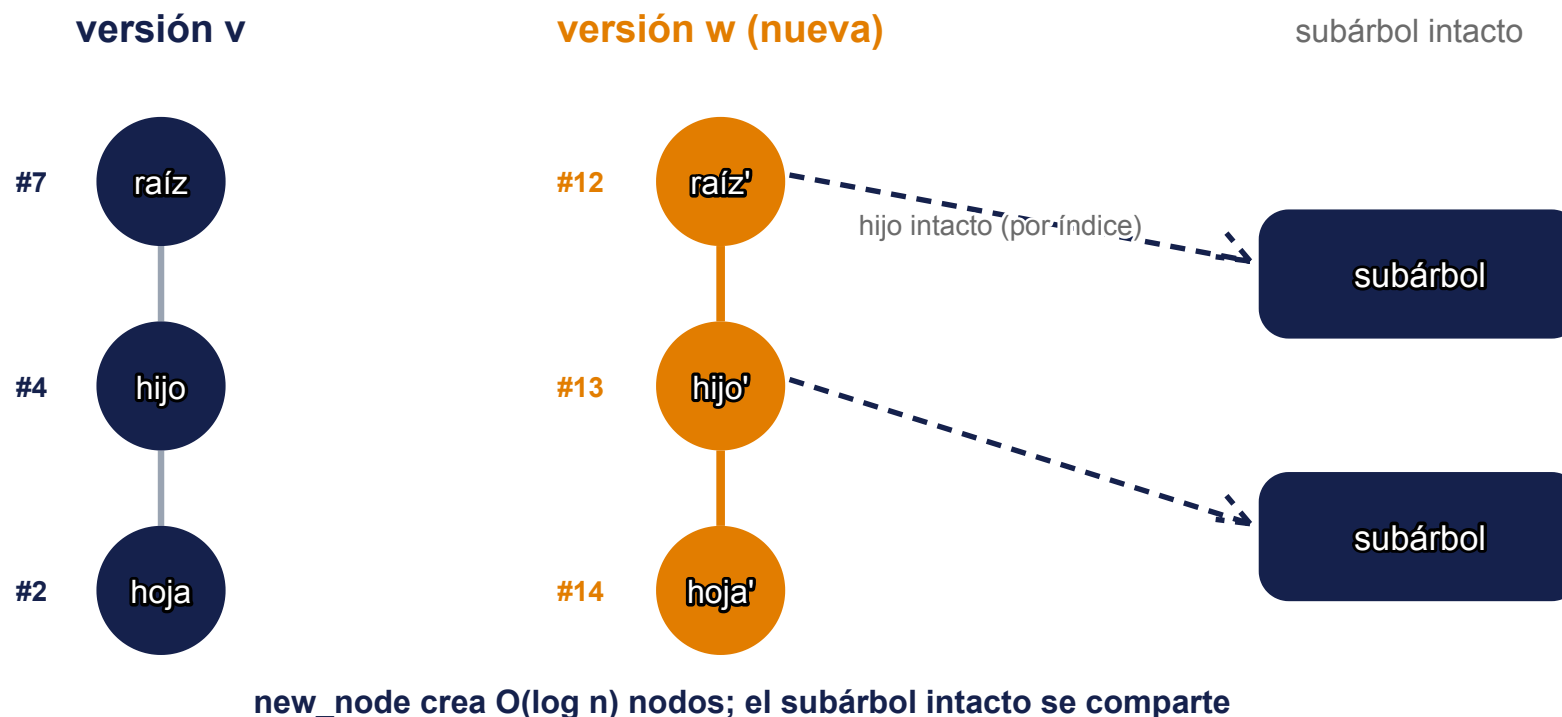
```
int build(int l, int r) {  
    if (l == r) return new_node(a[l]);  
    int m = (l + r) / 2;  
    int left  = build(l, m);  
    int right = build(m + 1, r);  
    return new_node(pool[left].val + pool[right].val, left, right);  
}
```

$2n - 1$ nodos para la versión inicial.



Update — Creando nueva versión

Recibimos la raíz de la versión base y devolvemos la nueva raíz. El hijo no modificado se comparte.



Se crean $O(\log n)$ nodos nuevos (naranjos); el subárbol intacto se apunta tal cual.



Update — Creando nueva versión

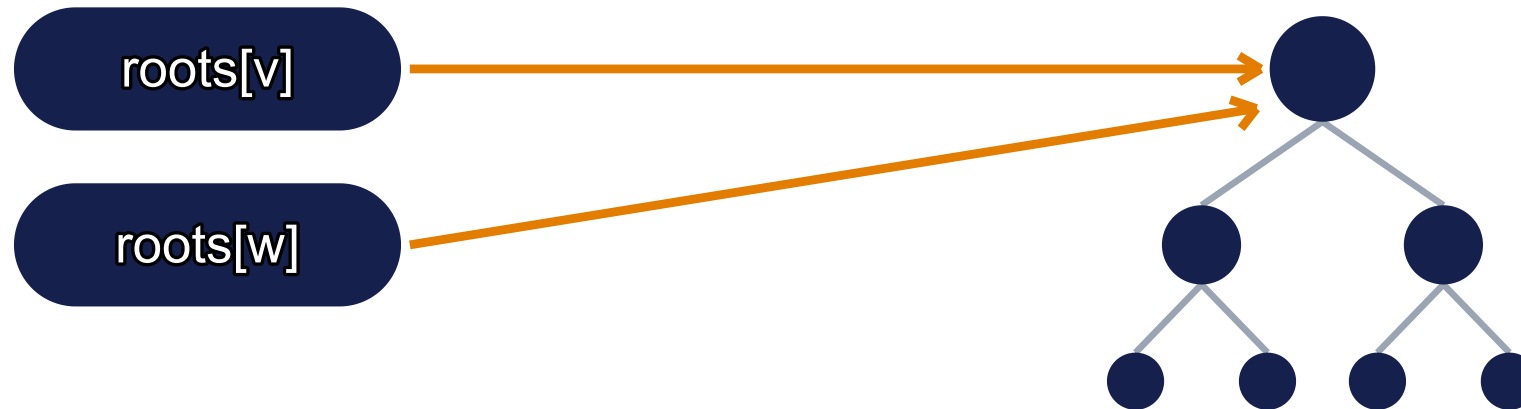
```
int update(int prev, int l, int r, int pos, int val) {  
    if (l == r) return new_node(val);  
    int m = (l + r) / 2;  
    int L = pool[prev].L, R = pool[prev].R;  
    if (pos <= m)  
        L = update(L, l, m, pos, val);          // nuevo hijo izquierdo  
    else  
        R = update(R, m + 1, r, pos, val);      // nuevo hijo derecho  
    return new_node(pool[L].val + pool[R].val, L, R);  
}
```

Se crean $O(\log n)$ nodos. El hijo no modificado se comparte.



Query — Idéntico al normal

Solo cambia de dónde parte: la raíz de la versión que queremos consultar.



misma rutina `query(...)`, distinta raíz de partida

Para consultar la versión `v` : `query(roots[v], 0, n-1, ql, qr)` .



Query — Idéntico al normal

```
int query(int node, int l, int r, int ql, int qr) {  
    if (qr < l || r < ql) return 0;  
    if (ql <= l && r <= qr) return pool[node].val;  
    int m = (l + r) / 2;  
    return query(pool[node].L, l, m, ql, qr)  
        + query(pool[node].R, m + 1, r, ql, qr);  
}
```

Para consultar la versión `v`: `query(roots[v], 0, n-1, ql, qr)`.



	Tiempo	Nodos nuevos
Build	$O(n)$	$2n - 1$
Update	$O(\log n)$	$\log_2(n) + 1$
Query	$O(\log n)$	0

Memoria total: $O(n + Q \log n)$, donde Q = cantidad de updates.

Para $n = 10^5$ y $Q = 10^5$: $\rightarrow \sim 2 \times 10^6$ nodos. Asignar entre $n * 20$ y $n * 40$ suele ser seguro.



Aplicación clásica — K-ésimo menor

K-ésimo menor en un subarray $[l, r]$ (sin updates): segment tree sobre valores, versión i = insertar $a[i]$ en la versión $i-1$.

valores	1	2	3	4	5	6	
ver[r]	2	1	3	0	2	1	
ver[l-1]	1	0	1	0	1	0	
= dif	1	1	2	0	1	1	
cuenta en $[l, r] = \text{ver}[r] - \text{ver}[l-1]$							

descender

↓

$k \leq \text{cntL} ? \text{izq} : \text{der}$

Para consultar $[l, r]$: restar la versión $l-1$ de la versión r (cuenta por prefijos).



Aplicación clásica — K-ésimo menor

```
int kth(int vl, int vr, int l, int r, int k) {  
    if (l == r) return l;  
    int m = (l + r) / 2;  
    int cntL = pool[pool[vr].L].val - pool[pool[vl].L].val;  
    if (k <= cntL)  
        return kth(pool[vl].L, pool[vr].L, l, m, k);  
    else  
        return kth(pool[vl].R, pool[vr].R, m + 1, r, k - cntL);  
}
```

`cntL` = cuántos valores del subárbol izquierdo caen en `[l, r]`, por resta de prefijos.

Tipos de persistencia

1 · Seg Tree

2 · Path Copying

3 · Tipos

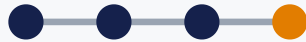
4 · Amortización

5 · Queue



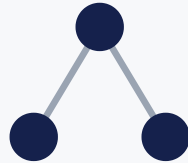
Tipos de Persistencia

No toda persistencia es igual. Hay cuatro niveles, según **qué versiones** se pueden modificar y si se pueden **combinar** (merge).



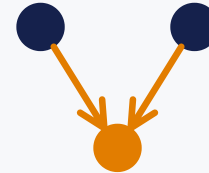
Parcial

solo la ultima; linea



Completa

cualquiera; arbol



Confluente

merge; DAG



Funcional

inmutable; arbol



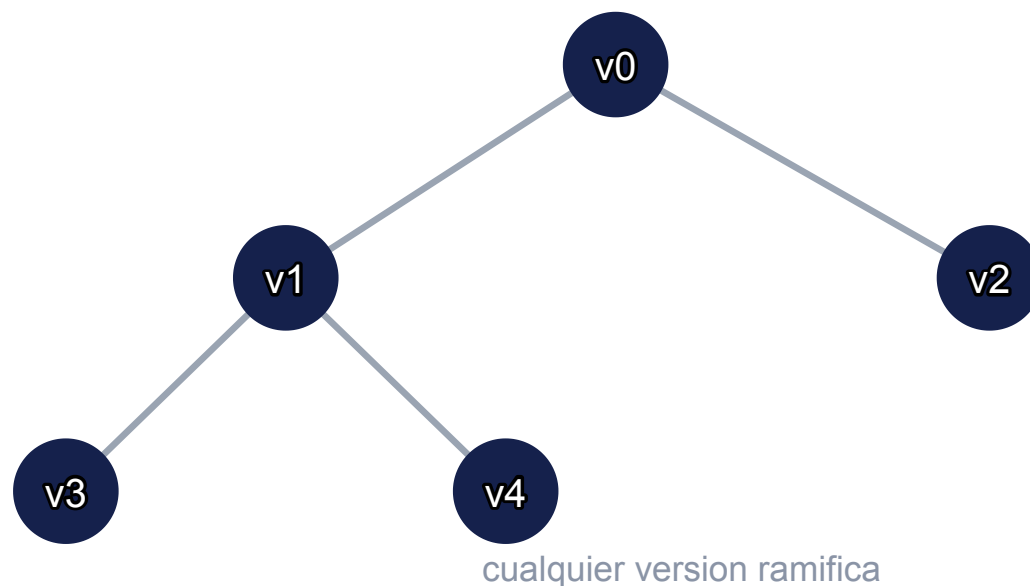
Persistencia Parcial



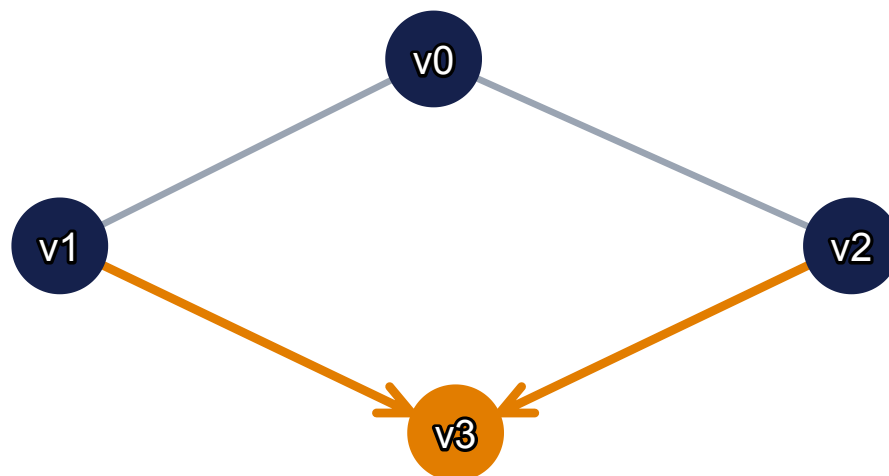
- Solo se puede modificar la **versión más reciente**; se pueden consultar todas.
- El timeline es una **línea recta**: $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow \dots$
- Ejemplo: undo/redo limitado, queries offline con rollbacks.



Persistencia Completa

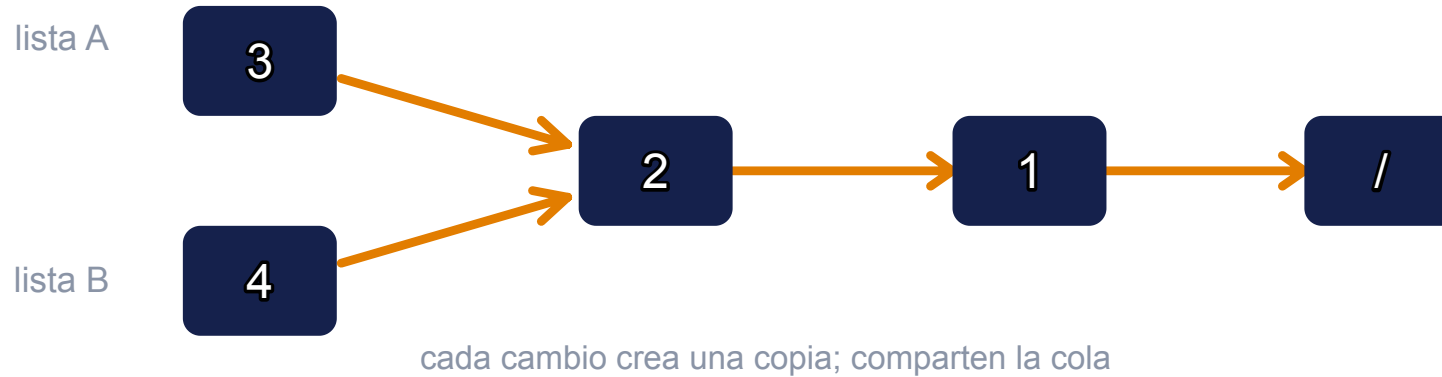


- Se puede modificar **cualquier versión**: cada update desde v_i ramifica creando v_j .
- El timeline es un **árbol** de versiones.
- Nuestro segment tree con path copying es **completamente persistente**.



combinar dos versiones $\rightarrow$ DAG

- Además de modificar cualquier versión, se pueden **combinar** dos versiones en una nueva.
- El timeline forma un **DAG** (Directed Acyclic Graph); el overhead por operación crece con $e(v)$.
- **Problema abierto:** ¿se puede lograr $O(\log n)$ de overhead en general?



- Las estructuras son **inmutables por diseño**; cada "modificación" crea una nueva copia.
- Ejemplo: listas enlazadas en Haskell/OCaml.
- El path copying es esencialmente una técnica funcional aplicada a estructuras mutables.

Persistencia y amortización

1 · Seg Tree

2 · Path Copying

3 · Tipos

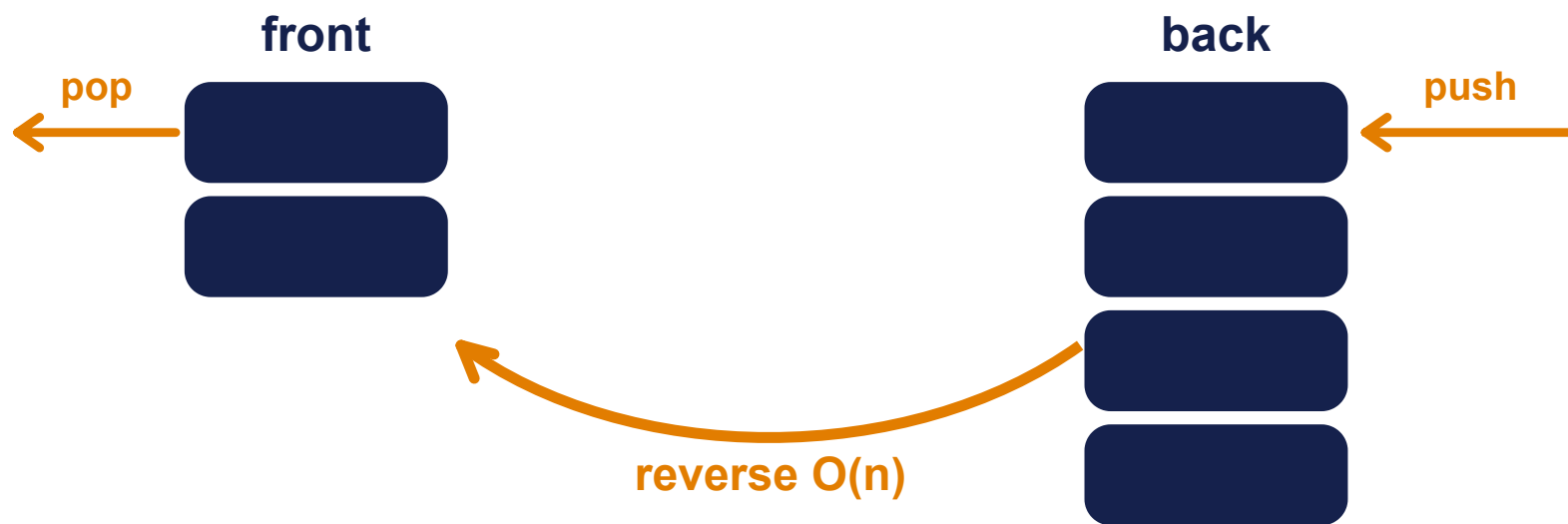
4 · Amortización

5 · Queue



Persistencia y Análisis Amortizado

No se llevan bien. El análisis amortizado asume que los **créditos** (o potencial) se gastan una sola vez.



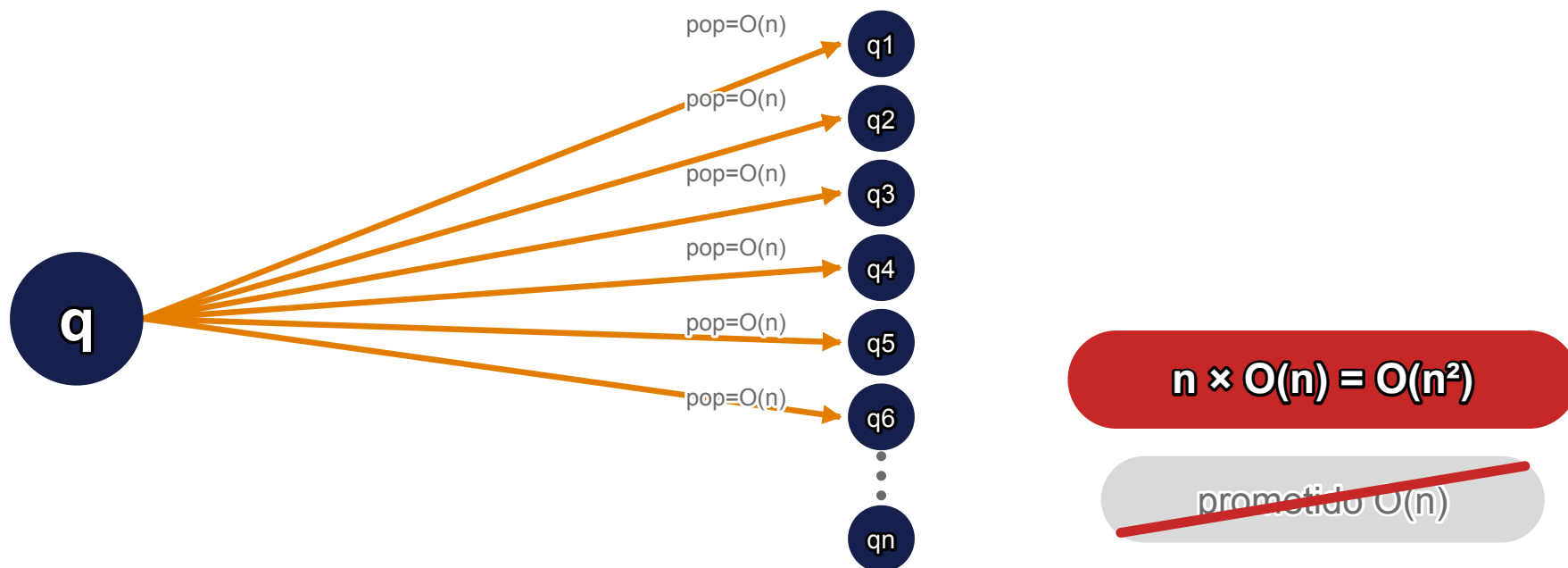
pop es $O(1)$ amortizado, pero $O(n)$ al invertir

Ejemplo — Cola con dos listas (two-list queue): `push` es $O(1)$ amortizado; `pop` es $O(1)$ amortizado... pero $O(n)$ en el peor caso, cuando hay que invertir la lista trasera.



El desastre

Guardas una versión justo antes del pop caro y ejecutas el mismo pop n veces desde q :



los creditos ya se gastaron en el primer pop

```
q1 = pop(q)    // O(n) - invierte la lista
q2 = pop(q)    // O(n) - ¡misma inversión desde q!
...
qn = pop(q)    // O(n)
```



amortizado

se rompe con persistencia

vs

worst-case $O(\log n)$

path copying: funciona siempre

Para estructuras de datos persistentes, se necesitan cotas **worst-case**, no amortizadas.

El **path copying** del segment tree persistente da **$O(\log n)$ worst-case** por operación. No depende de créditos ni potencial: funciona siempre.

Persistent Queue

1 · Seg Tree

2 · Path Copying

3 · Tipos

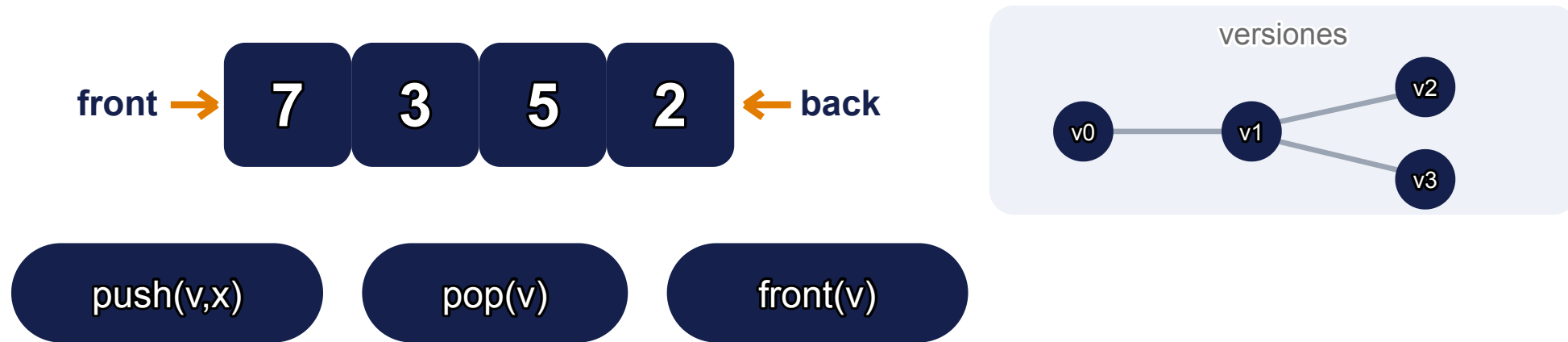
4 · Amortización

5 · Queue



Problema G — Persistent Queue

Implementar una cola donde cada operación crea una nueva versión:



cada operacion crea una **NUEVA version**

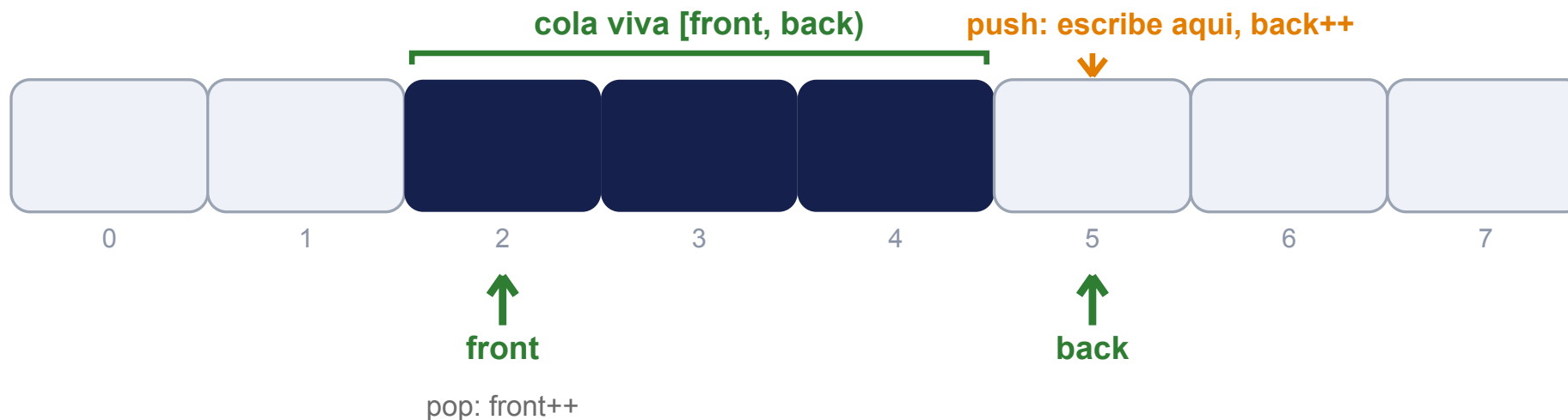
- `push(v, x)` → versión `w` : mete `x` al final de la cola en la versión `v`
- `pop(v)` → versión `w` : elimina el frente de la cola en la versión `v`
- `front(v)` → `x` : devuelve el elemento al frente de la versión `v`

Hasta 5×10^5 operaciones.



Solución con Segment Tree Persistente

Idea: modelar la cola como un array con dos punteros `[front, back)`, usando el ST persistente como el array de posiciones.



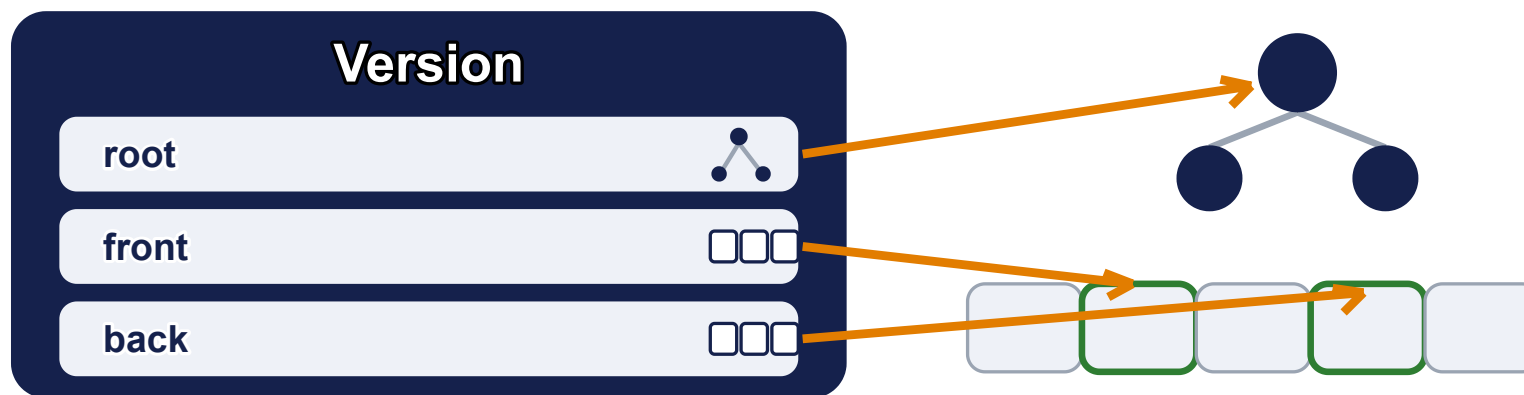
el array de posiciones es un segment tree persistente

- `push` : escribe en `pos[back]` y avanza `back` · `pop` : avanza `front` (no borramos)
- `front` : lee `pos[front]`



Estructura auxiliar

Para cada versión guardamos los punteros y la raíz del ST.



una version = raíz del ST + punteros front/back

El segment tree opera sobre el rango `[0, MAX_OPS]` (máximo de pushes posibles).



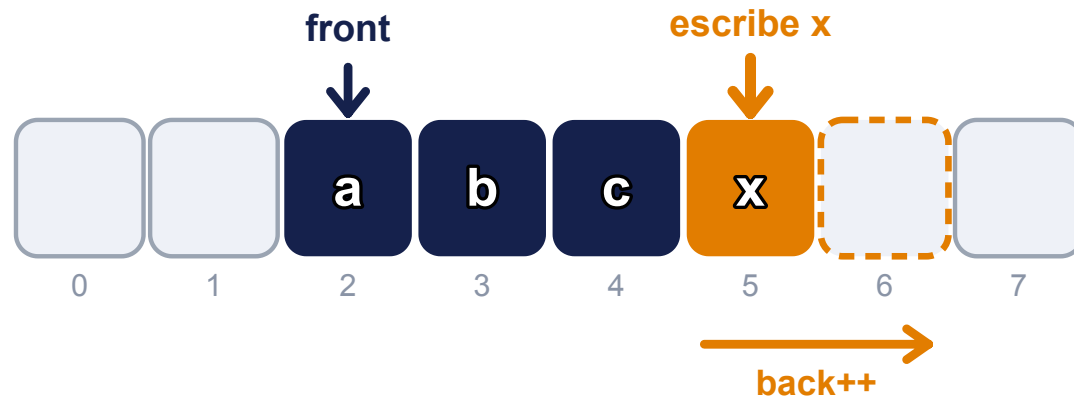
```
struct Version {  
    int root;    // raíz del segment tree para esta versión  
    int front;   // índice del primer elemento  
    int back;    // índice donde se insertará el próximo elemento  
};  
Version ver[MAX_VERSIONS];
```

El segment tree opera sobre el rango `[0, MAX_OPS]` (máximo de pushes posibles).



Push(v, x) $\rightarrow$ nueva versión w

Escribimos x en la posición `cur.back` (un update persistente) y avanzamos `back`.



update persistente
 $\rightarrow$ nueva raíz

push: 1 update $O(\log N)$, nueva versión



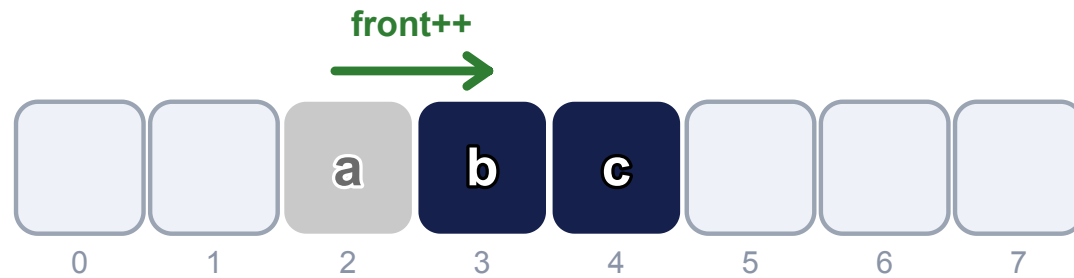
Push(v, x) → nueva versión w

```
int push(int v, int x) {  
    Version cur = ver[v];  
    // Escribir x en la posición cur.back  
    int new_root = update(cur.root, 0, MAX_OPS, cur.back, x);  
    ver[++vcnt] = {new_root, cur.front, cur.back + 1};  
    return vcnt;  
}
```



Pop(v) → nueva versión w

Solo avanzamos el puntero `front` : no tocamos el segment tree.



El segment tree NO cambia
(misma raíz)

pop: $O(1)$, el dato sigue ahí pero fuera de $[front, back)$

El dato sigue ahí, pero queda fuera del rango `[front, back)` .



Pop(v) → nueva versión w

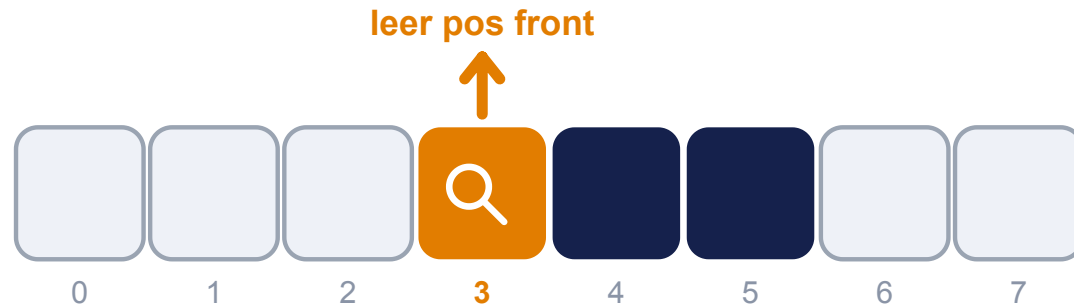
```
int pop(int v) {  
    Version cur = ver[v];  
    // "Eliminar" el frente: solo avanzamos el puntero  
    ver[++vcnt] = {cur.root, cur.front + 1, cur.back};  
    return vcnt;  
}
```

No tocamos el segment tree. El dato sigue ahí, pero queda fuera del rango [front, back) .



Front(v) → elemento al frente

Query puntual en la hoja `cur.front`.



query puntual
en la hoja front

o guardarlo en la versión para $O(1)$

Alternativamente, guardamos el valor en `Version` al hacer push y lo consultamos en $O(1)$.



Front(v) → elemento al frente

```
int front(int v) {  
    Version cur = ver[v];  
    // Leer la posición cur.front (es una hoja del segment tree)  
    return query(cur.root, 0, MAX_OPS, cur.front, cur.front);  
}
```

Query puntual en $O(\log n)$. Alternativamente, guardamos el valor en `Version` al hacer push y lo consultamos en $O(1)$.



Complejidad final

Operación	Tiempo	Nodos nuevos
push	$O(\log N)$	$O(\log N)$
pop	$O(1)$	0
front	$O(1)^*$	0

* $O(1)$ si guardamos el valor en la metadata de la versión; $O(\log N)$ si consultamos al segment tree.

Espacio total: $O(N \log N)$ donde N = cantidad de operaciones.



- **Segment tree normal:** $O(\log n)$ por operación, array implícito, sin historia
- **Segment tree persistente:** path copying, $O(\log n)$ worst-case, $O(n + Q \log n)$ memoria
- **Tipos de persistencia:** parcial $\rightarrow$ completa $\rightarrow$ confluente $\rightarrow$ funcional
- **Amortización + persistencia = $\times$** — usar cotas worst-case
- **Persistent Queue:** modelar con ST persistente + punteros front/back



- [USACO Guide — Persistent Data Structures](#)
- [Neelmishra — Persistent Segment Tree](#)
- [MIT 6.851 — Advanced Data Structures \(Lecture 1\)](#)
- [Cornell CS 3110 — Amortized Analysis & Persistence \(§9.3.4\)](#)
- [youkn0wwho.academy — Persistent Segment Tree Topic List](#)

¿Preguntas?

Gracias por su atención